# Exam Kit: Lectures 1 to 4

Introduction to compilers and lexical analysis. A 100-point practice exam with answer boxes to fill

## How to use this kit

This is a full practice exam on Lectures 1 to 4 only (introduction to compilers; tokens and regular expressions; automating lexical analysis; lexical analysis in practice). Nothing from the parsing lecture appears. The weights copy the style of the course's mock exams: a long multiple-choice part where every answer must be justified in one line, short questions on the compiler pipeline, construction problems on automata and lexers, and a true/false sweep.

| Part | Content | Points |
|---|---|---|
| A | 20 multiple-choice questions, one line of justification each | 40 |
| B | 5 short questions on the compiler pipeline (Lecture 1) | 15 |
| C | 12 construction problems on regular expressions, automata and lexers (Lectures 2 to 4) | 35 |
| D | 20 true/false statements | 10 |
| | Suggested time: 2 hours 30 minutes | 100 |

A version with full solutions under every question is *Exam_Kit_Lectures_1-4_Solutions.pdf*; work the whole paper first. Every automaton, scan trace, regular-expression equivalence and program output in this kit was checked by program (`check_lex.py`, `flex` and `cc`). Conventions: $\varepsilon$ is the empty string, `D` stands for `[0-9]`, `L` for `[a-zA-Z]`, and Thompson counts use **convention B** (concatenation merges two states) unless a question says otherwise.

## Part A: Multiple choice (20 × 2 points)

Choose one answer and justify it in one line. A correct letter without a justification earns one point.

**A1. Compiled or interpreted?**

`javac` turns `.java` files into `.class` files, which the JVM executes. Which statement is accurate?

(A) Java is an interpreted language, so `javac` is not a compiler.
(B) `javac` is a compiler whose target language is JVM bytecode; the JVM then interprets that bytecode and JIT-compiles the hot parts to native code.
(C) `javac` produces native machine code for the machine it runs on.
(D) The JVM is an assembler.

**A2. When to compile**

A 200-line data-cleaning script will be run exactly once, over a 50 GB input. Which choice is best justified?

(A) Interpret it: it runs once, and compile time would be wasted.
(B) Compile it (or JIT it): the single run is enormous, so execution time dominates compile time by far.
(C) Interpret it: interpreters are always faster on large inputs.
(D) Neither matters: both take the same total time.

### A3. Front ends and back ends

A vendor supports 4 source languages on 3 target machines. With a shared intermediate representation the number of translator components to write is

(A) 12 (B) 7 (C) 4 (D) 3

### A4. Which phase?

The statement `int n = "seven";` is rejected by

(A) the lexer (B) the parser (C) semantic analysis (D) the linker

### A5. Whose message is it?

`undefined reference to ‘sqrt’` is reported by

(A) the lexer (B) semantic analysis (C) the assembler (D) the linker

### A6. Parentheses

In `(a + b) * c`, the parentheses are

(A) kept as nodes of the AST so that code generation knows the order;
(B) discarded by the lexer as whitespace;
(C) used by the parser to shape the tree, then dropped, because the tree shape already encodes the grouping;
(D) replaced by temporaries during optimisation.

### A7. The most important optimisation

Lecture 1's `findIndex` example went from 18 memory accesses per iteration to none. The transformation responsible is

(A) constant folding (B) register allocation (C) dead-code elimination (D) inlining

### A8. The Chomsky hierarchy and the phases

Which pairing is wrong?

(A) Lexing: regular languages, finite automata.
(B) Parsing: context-free languages, pushdown automata.
(C) “Every variable is declared before use”: context-free, so it is checked by the parser.
(D) “This loop terminates”: undecidable in general, so the optimiser approximates conservatively.

### A9. Dead code

An optimiser is asked to delete unreachable statements. Because reachability is undecidable, a correct optimiser

(A) refuses to delete anything;
(B) deletes a statement only when it can *prove* it unreachable, and otherwise keeps it;
(C) deletes a statement unless it can prove it reachable;
(D) runs the program to find out.

### A10. Tokens and lexemes

For `count = count + 1`, with the usual rules, which statement is true?

(A) There are 5 tokens and the two occurrences of `count` are the same token with the same attribute.

(B) There are 3 tokens, because `count` appears twice.
(C) There are 5 tokens, and the lexer already decides that both `count`s refer to the same variable.
(D) There are 7 tokens, counting the spaces.

### A11. A regular expression

Which expression denotes exactly the strings over $\{a, b\}$ with an *even* number of $a$'s?
(A) $(aa \mid b)^*$ (B) $(b \mid ab^*a)^*$ (C) $b^*(ab^*a)^*$ (D) $(a \mid b)^*aa(a \mid b)^*$

### A12. Counting Thompson states

Under convention B, Thompson's construction for $(a \mid b)^*abb$ produces
(A) 9 states (B) 10 states (C) 11 states (D) 14 states

### A13. Subset construction

An NFA with $n$ states is converted to a DFA by the subset construction. Which is true?
(A) The DFA always has at most $n$ states.
(B) The DFA can have up to $2^n$ states, and some languages ("the $k$-th symbol from the end is $a$") really need $2^k$.
(C) The DFA always has exactly $2^n$ states.
(D) The blow-up disappears after minimisation.

### A14. Minimal DFAs

Two students minimise the same DFA and obtain automata with different state names but the same number of states. Which is true?
(A) At most one of them is right.
(B) Both can be right: the minimal DFA is unique up to renaming of states.
(C) Both are wrong, since minimisation always yields fewer states than the NFA.
(D) Minimal DFAs are not unique, so nothing can be concluded.

### A15. Longest match

Rules, in order: `IF = if`, `IN = in`, `INT = int`, `ID = L(L|D)*`. The input `integer` is lexed as
(A) `IN ID(teger)` (B) `INT ID(eger)` (C) `ID(integer)` (D) `IF` then an error

### A16. Rule order

With the same rules, `int` is lexed as `INT` and not `ID`, because
(A) keywords are always preferred to identifiers by every lexer generator;
(B) both rules match 3 characters, and the tie is broken by the earlier rule;
(C) `ID` cannot match `int`;
(D) `INT` is longer than `ID`.

### A17. The dot

In `flex`, a rule `"//".*` for line comments works because
(A) `.` matches any character including newline, and the comment ends at end of file;
(B) `.` matches any character *except* newline, so the match stops at the end of the line;
(C) `*` is non-greedy by default;

(D) `//` is a special token for comments.

### A18. The classic hand-written bug

A hand-written lexer reads identifiers with `while (isIdChar(c = read()))`. On `sum+1` it produces `ID(sum) NUM(1)`. The cause is

(A) `+` is not a token in this language;
(B) the loop consumed the `+` to discover that the identifier was over, and never gave it back;
(C) `isIdChar` is wrong;
(D) longest match prefers `NUM`.

### A19. Nested comments

Properly nested block comments cannot be recognised by a regular expression because

(A) comments may contain newlines;
(B) a finite automaton cannot count an unbounded nesting depth; the language is the shape of balanced parentheses;
(C) `/*` contains a metacharacter;
(D) they can, with `"/*"(.|\n)*"*/"`.

### A20. From source to a running program

Put in order the tools that turn `main.c` and `util.c` into a running process, and name what resolves the call from `main` to a function defined in `util.c`.

(A) compiler, linker, assembler, loader; the assembler.
(B) compiler, assembler, linker, loader; the linker.
(C) assembler, compiler, loader, linker; the loader.
(D) compiler, assembler, loader; the compiler, since it sees both files.

## Part B: The compiler pipeline (5 × 3 points)

### B1. Sort the errors

For each message, name the stage that produces it (lexer, parser, semantic analysis, linker, run time) and say in a few words why that stage and not an earlier one.

1. `x = y +* 2;`
2. `"unterminated string`
3. `return 3 # 4;`
4. `undeclared identifier ‘tmp’`
5. `undefined reference to ‘parse_header’`
6. `integer division by zero`

### B2. The intermediate representation

A company maintains compilers for C, Rust and Swift targeting x86, ARM and RISC-V, built around one shared IR.

**(a)** How many front ends and back ends exist, and what would the count be without the IR?

**(b)** A new optimisation (loop unrolling) is to be added. Where in the pipeline does it go, and how many copies of it must be written? Why?

**(c)** The company adds WebAssembly as a target. What has to be written, and what does not?

### B3. Compile, interpret, or both

For each situation say whether you would compile ahead of time, interpret, or use a JIT, and give the deciding reason in one sentence.

**(a)** A spreadsheet formula typed by a user and evaluated once.

**(b)** A game engine's physics loop shipped to millions of players.

**(c)** A web browser's JavaScript engine running an unknown page.

### B4. Where in the hierarchy

Complete the table: for each property, give the Chomsky type (or "undecidable"), the machine that recognises it, and the compiler phase that deals with it.

| Property or language | Type | Machine | Phase |
|---|---|---|---|
| Floating-point literals | | | |
| Properly nested `begin`/`end` blocks | | | |
| Every `break` is inside a loop | | | |
| "This function never returns `null`" | | | |

### B5. Looking inside the pipeline

Match each command or tool to what it produces, and name the stage where `undefined reference` and `expected ';'` are reported.

```
gcc -E   gcc -S   gcc -c   as   ld   objdump -d
```

## Part C: Constructions (35 points)

### C1. Write the regular expressions (4 points) (⋆)

Give a regular expression for each token. Use `D` for a digit, `L` for a letter, and ordinary regex operators.

**(a)** C identifiers: letters, digits and underscores, not starting with a digit.

**(b)** Hexadecimal literals such as `0x1F`, `0XdeadBEEF`, with at least one hex digit.

**(c)** Decimal literals with a point, where at least one side of the point has a digit (`3.`, `.5`, `3.5` but not `.`), and an optional exponent `e-12`, `E7`.

**(d)** String literals: double quotes around any characters, where `\"` is an escaped quote and `\\` an escaped backslash, and a newline is not allowed inside.

**(e)** Binary strings with an even number of `1`s.

**(f)** Strings over $\{a, b\}$ that never contain two consecutive $b$'s.

## C2. Token sequence with longest match (3 points) (⋆⋆)

Rules in priority order: `IF = if`, `LE = <=`, `EQ = ==`, `LT = <`, `ASSIGN = =`, `FLOAT = D+.D+`, `NUM = D+`, `ID = L(L|D)*`. Whitespace separates and is dropped; a character that starts no token is reported as an error and skipped.

**(a)** Give the token sequence for `if1<=x2=3.14.5<==y` (no spaces in the input).

**(b)** For `iff<==3.<=x`, give the sequence and say at which token the scanner has to push back *two* characters.

## C3. Thompson's construction (3 points) (⋆⋆)

Let $r = (ab \mid b)^*\, a$.

**(a)** Parse $r$: give the tree, and count leaves, unions, stars and concatenations.

**(b)** Build $N(r)$ with convention B. Give every edge, the start and the accepting state, and check the state and edge counts against the formulas.

**(c)** What are the counts under convention A?

## C4. Subset construction and minimisation (4 points) (⋆⋆)

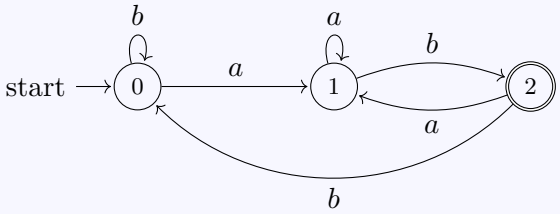
Using the NFA of C3:

**(a)** Run the subset construction. Name the DFA states $A, B, C, \ldots$ in the order you discover them and give each one's NFA set.

**(b)** Minimise the DFA by table filling. Give the final automaton and describe its language in words.

## C5. State elimination, DFA to regular expression (3 points) (⋆⋆)

The DFA $M$ over $\{a, b\}$ has start state 0 and accepting state 2:




**(a)** Describe $L(M)$ in one sentence.

**(b)** Apply state elimination (add a new final state, remove 1, then remove 2) and give the resulting expression. Then give the shortest expression you can for the same language.

## C6. Brzozowski derivatives (2 points) (⋆⋆)

Build the DFA of $R = a^*ba^*$ directly by derivatives: compute $\partial_a$ and $\partial_b$ of every expression you reach, say which are nullable, and draw the automaton. Do the same for $(ab)^*$.

## C7. Lower bounds and non-regularity (4 points) (⋆⋆⋆)

**(a)** Let $L = \{w \in \{a,b\}^* : \#a(w) - \#b(w) \equiv 0 \pmod 3\}$. Show that every DFA for $L$ needs at least 3 states, and give one with exactly 3.

**(b)** Show with distinguishable strings that $\{a^n b^m : n \geq m\}$ is not regular.

**(c)** Show with the pumping lemma that the palindromes over $\{a,b\}$ are not regular.

## C8. The combined DFA and a scan with pushback (4 points) (⋆⋆⋆)

Token rules, in priority order: `P1 = a`, `P2 = ab+`, `P3 = b*a`.

**(a)** Build the combined DFA (one automaton for all three rules) and label every accepting state with the token it reports.

**(b)** Which accepting state's label is decided by rule order rather than by length? Explain.

**(c)** Scan `abbabaa` with longest match. For each token give the states visited, the last accepting position, the pushback, and the token. How many characters are read in total?

**(d)** What does the scanner do on the input `bbb`?

## C9. A hand-written lexer (2 points) (⋆⋆)

The C program below is meant to print `NUM`, `ID`, `EQ` (`==`), `ASSIGN` (`=`) and `OTHER` tokens.

```
int c;
while ((c = getchar()) != EOF) {
    if (isdigit(c)) {
        int v = 0;
```

```
        while (isdigit(c)) { v = v * 10 + (c - '0'); c = getchar(); }
        printf("NUM(%d) ", v);
    } else if (isalpha(c)) {
        char buf[32]; int n = 0;
        buf[n++] = c;
        while (isalnum(c = getchar())) buf[n++] = c;
        buf[n] = 0;
        printf("ID(%s) ", buf);
    } else if (c == '=') {
        if (getchar() == '=') printf("EQ ");
        else printf("ASSIGN ");
    } else if (c == ' ' || c == '\n') {
    } else printf("OTHER(%c) ", c);
}
```

**(a)** Give the output for the input `x1=2 y==3+x` and the correct output.

**(b)** Name the bugs and fix them with `ungetc`.

## C10. Predict flex (2 points) (⋆⋆)

```
%%
"if"            printf("IF ");
"i"             printf("I ");
[a-z]+          printf("ID(%s) ", yytext);
[0-9]+          printf("INT(%s) ", yytext);
[0-9]+"."[0-9]* printf("FLT(%s) ", yytext);
"."             printf("DOT ");
"=="            printf("EQ ");
"="             printf("AS ");
[ \t\n]         ;
%%
```

**(a)** Give the output for `if ifx i 3.14 3. .5 x==1 x=== 7 iff`.

**(b)** Give the output for `i@f`.

**(c)** The rule `"i"` is moved below `[a-z]+`. What does `flex` say, and what changes in the output of (a)?

## C11. Lexer states (2 points) (⋆⋆)

```
%x STR COM
%{
int depth = 0; char buf[256]; int n;
%}
%%
\"            { n = 0; BEGIN(STR); }
<STR>\\n      { buf[n++] = '\n'; }
<STR>\\\"     { buf[n++] = '"'; }
<STR>\\.      { buf[n++] = yytext[1]; }
<STR>\n       { printf("ERR "); BEGIN(INITIAL); }
```

```
<STR>\"        { buf[n] = 0; printf("STR(%s) ", buf); BEGIN(INITIAL); }
<STR>.         { buf[n++] = yytext[0]; }
"/*"           { depth = 1; BEGIN(COM); }
<COM>"/*"      { depth++; }
<COM>"*/"      { if (--depth == 0) BEGIN(INITIAL); }
<COM>.|\n      ;
[a-z]+         printf("ID(%s) ", yytext);
[ \t\n]        ;
.              printf("OTHER(%c) ", yytext[0]);
%%
```

**(a)** Output for `a "x\"y" /* b /* c */ d */ e`, with the state after each rule fires.

**(b)** Output for the two lines `p "abc` and `q`.

**(c)** Delete the rule `<COM>"/*"`. What is the output of (a) now?

**(d)** Why must `<STR>\\.` come after the two rules above it, and why does `<STR>\n` fire at all given the rule `<STR>.`?

### C12. A comment regex with a two-character opener and a three-character closer (2 points) (⋆⋆⋆)

HTML comments start with `<!--` and end at the first `-->`. Let $x$ stand for any character other than `-` and `>`.

**(a)** Give a single regular expression for a complete comment.

**(b)** Draw the DFA and say what each state remembers.

## Part D: True or false (20 × 0.5 points)

Answer each statement and give the one-word reason.

### D1 to D20

1. A compiler's output must be machine code.
2. Semantic analysis is a separate tree walk because its constraints are not context-free.
3. The lexer decides whether two occurrences of a name refer to the same variable.
4. The parser runs in $O(n^3)$ for every grammar.
5. Every regular language is accepted by some DFA with a unique minimal number of states.
6. Every NFA can be converted to an equivalent DFA.
7. Every DFA with $n$ states accepts a language that some NFA accepts with fewer than $n$ states.
8. $\varepsilon$-closure must be taken after every move in the subset construction.
9. In a DFA, a single failed path means the string is rejected.
10. In an NFA, a single failed path means the string is rejected.
11. Thompson's construction yields an NFA with at most two outgoing edges per state.
12. A regular expression and its minimal DFA may both be exponentially larger than each other, in different cases.

13. Rule order in a lexer specification can override longest match.
14. A lexer with pushback can read $\Theta(n^2)$ characters on an input of length $n$.
15. `.` in flex matches a newline.
16. Nested block comments can be handled by a regular expression if it is long enough.
17. A keyword that appears inside a comment is still recognised by the keyword rule.
18. Rice's theorem implies that an optimiser can never remove any dead code.
19. Python's indentation-based blocks are lexed with the help of a stack of indentation widths.
20. A minimal DFA may have a dead state.

## Where to revise

| Questions | Chapter and section |
|---|---|
| A1–A9, A20, B1–B5, D1–D4, D18 | *01 Introduction to Compilers*: §1.2 compilation vs interpretation, §3 the phases and front/back ends, §4 the Chomsky hierarchy, §6 classic questions |
| A10, A15–A18, C1, C2, C9, D13, D14 | *Lexical Analysis Complete*, Part I (§1–§4): tokens, hand-written lexers, token patterns, longest match and rule order |
| A11–A14, C3–C7, D5–D12, D20 | *Lexical Analysis Complete*, Part II: Methods 1 to 7 (Thompson, simulation, subset construction, minimisation, state elimination, derivatives, lower bounds and the pumping lemma) |
| C8, D14 | *Lexical Analysis Complete*, §13–§14: the combined DFA and longest match in a running lexer |
| A17, A19, C10–C12, D15–D17, D19 | *Lexical Analysis Complete*, §16 lexer states, the comment DFA and quirky languages, and §17 flex and JFlex |